

Traduction d'un Diagramme de Classe en Java

Aminata ZERBO/SABANE



A
+a1: String a2: String #a3: String -a4: String
+op1() +op2()

```

public class A {
    public    String a1;
    package   String a2;
    protected String a3;
    private   String a4;
    public void op1() {
        ...
    }
    public void op2() {
        ...
    }
}

```

Génération de Code

UML → Java



Traduction d'un Diagramme de Classe en Java

Ce chapitre nous permettra de voir quelques règles qui permettent de traduire un diagramme de classes en code Java

Ces règles sont à la base des outils de génération automatique de code à partir des diagrammes de classes

UML est indépendant de tout langage de programmation orienté objet

Mais UML partage avec ces langages une sémantique commune concernant les concepts orientés objet

Sémantique commune mais pas identique

UML # Java

Plusieurs concepts UML sont inexistants en Java dans les langages de programmation

Des artifices sont alors utilisés pour les représenter

Exemple : L'association

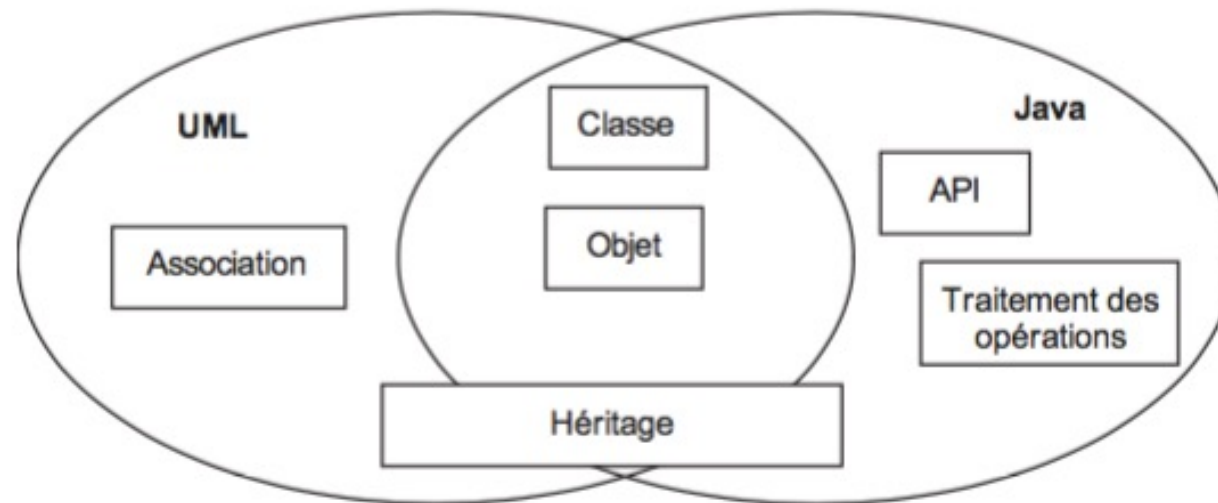
Il existe également des concepts de Java qui n'existent pas en UML

Exemple : Le corps des méthodes

Il existe des éléments communs à UML et Java qui n'ont pas la même signification ou portée

Exemple : L'héritage

UML # Java



Schématisation de la relation existant entre les
sémantiques UML et Java

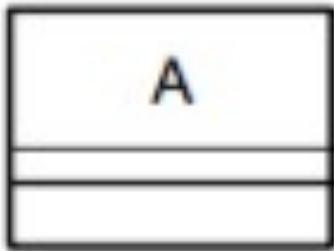
Règles de Correspondance UML → Java



Classe UML

À toute classe UML
doit correspondre une
classe Java

La classe Java porte le même
nom que la classe UML



```
public final class A {
```

```
    public A() {  
        super();
```

```
    ...
```

```
}
```

```
    protected void finalize() throws Throwable {  
        super.finalize();
```

```
    ...
```

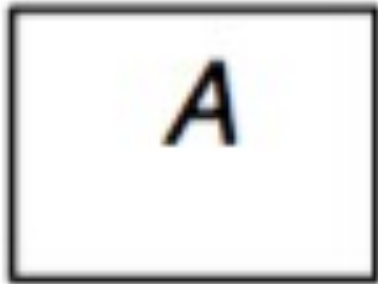
```
}
```

```
    ...
```

```
}
```


Classe UML

A chaque classe abstraite UML, doit correspondre une classe abstraite Java

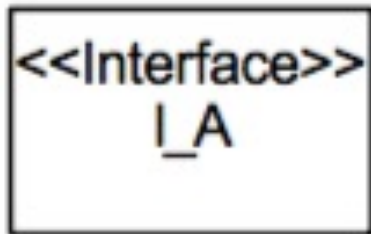


```
public abstract class A {  
    ...  
}
```

Interface UML

À toute interface UML doit correspondre une interface Java

L'interface Java porte le même nom que l'interface UML



```
public interface I_A {  
    ...  
}
```

Propriétés UML

À toute propriété d'une classe UML doit correspondre un attribut appartenant à la classe Java correspondant

- Le nom de l'attribut doit être le même que le nom de la propriété UML
- Le type de l'attribut doit être une correspondance Java du type de la propriété UML
- Si le nombre maximal de valeurs pouvant être portées par la propriété est supérieur à 1, l'attribut Java est un tableau.

Propriétés UML

À toute propriété d'une classe UML doit correspondre un attribut appartenant à la classe Java correspondant

- Le nom de l'attribut doit être le même que le nom de la propriété
- Le type de l'attribut doit être une correspondance Java du type de la propriété UML
- Si le nombre maximal de valeurs pouvant être portées par la propriété est supérieur à 1, l'attribut Java est un tableau.

Opérations UML

À toute opération d'une classe UML doit correspondre une opération appartenant à la classe Java correspondant à la classe UML

Les noms des opérations doivent être les mêmes

Paramètres d'Opérations UML

UML supporte quatre types de paramètres (in, out, in-out, return)

Java en supporte uniquement deux (in, return)

Si l'opération contient des paramètres out, in-out

→ Pas de génération de code Java possible

Paramètres d'Opérations UML

A chaque paramètre in de l'opération UML, correspond un paramètre de la méthode Java

Les noms des paramètres doivent être les mêmes dans l'opération UML et la méthode Java

Les types des paramètres doivent être une correspondance Java des types des paramètres UML

Si l'opération UML contient un paramètre return, la méthode Java doit définir un retour qui lui correspond

Sinon, la méthode Java retourne void

Example

A
+a1: String a2: String #a3: String -a4: String
+op1() +op2()

```
public class A {  
    public String a1;  
    package String a2;  
    protected String a3;  
    private String a4;  
  
    public void op1() {  
        ...  
    }  
    public void op2() {  
        ...  
    }  
}
```


Relation d'Association

Si une classe UML A est associée à une classe UML B

- Si l'association est navigable de A vers B
 - A doit contenir un attribut de type la classe B
 - Le nom de l'attribut est le nom du rôle de la classe B
 - Si le nombre maximal d'objets pouvant être reliés est supérieur à 1, l'attribut Java est un tableau

Relation d'Association

Si une classe UML A est associée à une classe UML B

- Si l'association n'est pas navigable
 - A doit contenir un attribut de type la classe B
 - B doit contenir un attribut de type la classe A
 - Si le nombre maximal d'objets pouvant être reliés est supérieur à 1, l'attribut Java est un tableau

Relation d'Association



```
public class A {
```

```
    public B m_Rb;
```

```
}
```

```
public class B {
```

```
    public A m_Ra;
```

```
}
```

Relation d'Association



```
public class A {  
  
    public B m_Rb;  
  
}
```

```
public class B {  
  
    public Vector m_Ra = new  
    Vector();  
  
}
```

Relation d'Association



```
public class A {
```

```
    public B m_Rb;
```

```
}
```

```
public class B {
```

```
    private A[] m_Ra = new A[5];
```

```
}
```

Relation d'Agrégation et de Composition

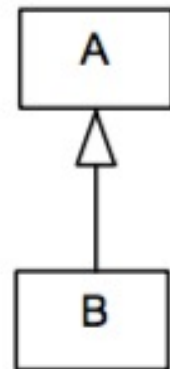
Les agrégations s'implémentent comme les associations

Une composition peut s'implémenter comme une association unidirectionnelle

Héritage entre Classes

Si une classe UML A hérite d'une autre classe UML B

→ il y a une relation d'héritage (extends en Java) entre les classes Java A et B correspondantes



```
public class B extends A {  
    ...  
}
```

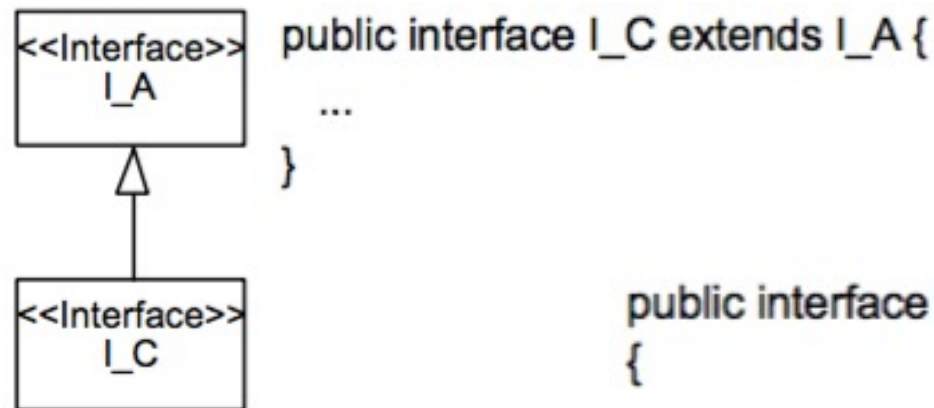
Si une classe UML hérite de plusieurs autres classes UML

→ impossible de générer du code Java

Héritage entre Interfaces

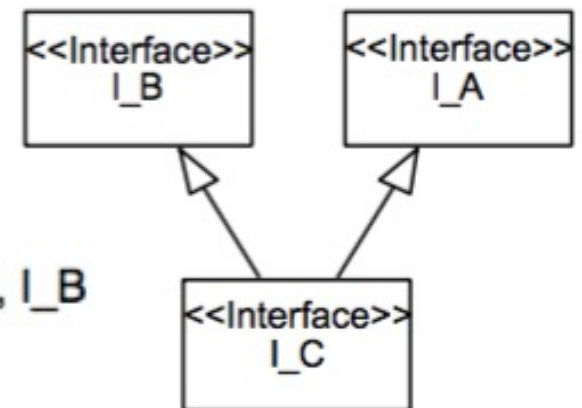
Si une interface UML IC hérite d'une autre interface UML IA

→ il y a une relation d'héritage (extends en Java) entre les interfaces Java IA et IB correspondantes



```
public interface I_C extends I_A {  
    ...  
}
```

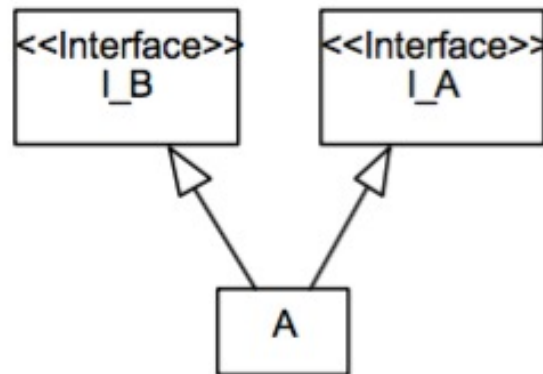
```
public interface I_C extends I_A, I_B  
{  
    ...  
}
```



Implémentation d'Interfaces

Si une classe UML A réalise une ou plusieurs interfaces UML

→ La classe Java A implemente les interfaces Java correspondantes



```
public class A implements I_A, I_B {  
    ...  
}
```

Import de Packages

Si un package UML P1 importe un autre package UML P2

→ Toutes les classes Java correspondant aux classes UML du package P1 doivent déclarer un import Java vers toutes les classes Java correspondant aux classes UML du package UML P2

Notes de Code

Si les opérations UML contiennent des notes de code Java
→ Ce code Java est recopié dans les méthodes Java correspondantes

Limites de la Génération de Code



Limites de la Génération de Code

La génération n'est pas toujours possible

Exemple : Héritage multiple

Les règles n'intègrent pas l'utilisation de l'API Java
(Remplacer un tableau par un arrayList)

Le code obtenu est un squelette
Pourquoi?

Références



Références

Modélisation objet avec UML, Pierre-Alain Muller

UML 2 pour les développeurs : Cours avec exercices corrigés, Xavier Blanc, Isabelle Mounier